

DOCUMENTATION TECHNIQUE

Nexus Games

Application web de gestion de collections de jeux vidéo

Stack technique: MongoDB - Node.js - Docker - Express.js - React

API source: RAWG

Projet PPROJ3 — 2025/2026

Sommaire

Sommaire.....	2
1. Procédure d'installation et prérequis	3
1.1 Obtenir une clé API RAWG.....	3
1.2 Installation de Docker.....	3
Windows	3
MacOs.....	3
Linux	3
1.3 Mise en place des variables d'environnement.....	3
Configuration Google OAuth.....	4
2. Guide de déploiement.....	5
2.1 Déploiement local (développement)	5
2.2 Architecture des conteneurs	5
2.3 Arrêt et maintenance.....	5
3. Justification des choix technologiques.....	6
3.1 Langage.....	6
3.2 Framework Backend	6
3.3 Framework Frontend.....	6
3.4 Base de données	6
3.5 Conteneurisation	6
3.6 Choix de l'api tierce.....	7
3.7 Authentification.....	7
4. Diagrammes UML.....	8
4.1 Diagramme de cas d'utilisation.....	8
4.2 Diagramme de séquence — Interaction avec l'API RAWG	9
5. Schéma de la base de données.....	10
5.1 Relations entre collections	10
5.2 Conception des models	11

1. Procédure d'installation et prérequis

Nexus Games nécessite Docker et une clé API RAWG pour fonctionner.
Suivez les étapes ci-dessous dans l'ordre.

1.1 Obtenir une clé API RAWG

L'application s'appuie sur la l'api RAWG pour récupérer les données des jeux.
Voici comment obtenir gratuitement une clé :

1. Visiter <https://rawg.io/apidocs>
2. Cliquer sur le bouton « Get API Key »
3. Se connecter ou créer un compte RAWG
4. Remplir le formulaire avec les informations suivantes :
 - Full name : votre nom
 - Site / App URL : `http://localhost:3000/`
 - Use Description : Use/Test a school project that use the RAWG API
5. La clé API est générée et affichée dans votre espace personnel
6. Copier cette clé : elle sera utilisée dans la configuration du fichier `.env`

1.2 Installation de Docker

Docker est le seul outil à installer sur votre machine. Il encapsule tous les services de l'application.

Windows

- Installer WSL (Windows Subsystem for Linux) : ouvrir le Microsoft Store et installer Ubuntu
- Télécharger Docker Desktop (éviter version ARM) depuis <https://docs.docker.com/desktop/setup/install/windows-install/>
- Suivre l'assistant d'installation
- Vérifier que la virtualisation est activée dans le BIOS

MacOs

- Télécharger Docker Desktop depuis <https://docs.docker.com/desktop/setup/install/mac-install/>
- Choisir la version adaptée à l'architecture de votre machine

Linux

- Suivre les instructions officielles depuis <https://docs.docker.com/desktop/setup/install/linux/>

Après l'installation, lancer Docker Desktop.

1.3 Mise en place des variables d'environnement

Toute la configuration sensible (clés, mots de passe) passe par un fichier `.env` placé à la racine du projet.

1. Cloner le dépôt depuis <https://github.com/Alban-E/P PROJ3>
2. Dupliquer le fichier « .env.example » présent à la racine du projet.
3. Renommer la copie en « .env »
4. Remplir les variables suivantes :

Variable	Exemple de valeur	Description
FRONTEND_PORT	3000	Port d'accès au frontend
BACKEND_PORT	5000	Port du serveur backend
MONGO_USER	admin	Nom d'utilisateur MongoDB
MONGO_PASS	MotDePasseMongo	Mot de passe MongoDB
GOOGLE_CLIENT_ID	xxx.apps.googleusercontent.com	ID client Google OAuth
GOOGLE_CLIENT_SECRET	GOCSPX-xxx	Secret Google OAuth
RAWG_API_KEY	votre_clé_rawg	Clé API RAWG (voir 1.1)
JWT_SECRET	une_chaine_aleatoire_longue	Clé de signature JWT
CREATE_ADMIN_ACCOUNT	MotDePasseAdmin	Mot de passe nécessaire pour créer un compte admin (voir fichier .env.example)

Configuration Google OAuth

Les variables GOOGLE_CLIENT_ID et GOOGLE_CLIENT_SECRET nécessitent une configuration sur la Google Cloud Console :

1. Se connecter sur <https://console.cloud.google.com/apis/dashboard>
2. Sélectionner « Identifiants » dans le menu de gauche
3. Cliquer sur « Créer un identifiant » → « ID client OAuth »
4. Choisir le type d'application « Application Web » et remplir le formulaire
5. Ajouter <http://localhost:3000> (votre frontend_port à la place de 3000) comme origine autorisée et <http://localhost:5000/auth/google/callback> (votre backend_port à la place de 5000) comme URI de redirection
6. Copier l'« ID client » et le « Code secret du client » dans le fichier « .env »

2. Guide de déploiement

2.1 Déploiement local (développement)

Une fois les prérequis installés et le fichier .env configuré :

1. S'assurer que Docker Desktop est lancé.
2. Ouvrir un terminal à la racine du dépôt cloné.
3. Exécuter la commande suivante :
 - `docker compose up`
4. Patienter le temps que Docker télécharge les images et démarre les conteneurs (première exécution plus longue).
5. Accéder à l'application dans un navigateur : <http://localhost:3000/> (remplacer 3000 par votre frontend_port)

2.2 Architecture des conteneurs

Docker Compose orchestre trois services indépendants :

- Frontend
 - Image : node:20-alpine
 - Port exposé : FRONTEND_PORT (3000)
 - Rôle : Interface React pour l'utilisateur
- Backend
 - Image : node:20-alpine
 - Port exposé : BACKEND_PORT (5000)
 - Rôle : API Rest via express.js
- MongoDB
 - Image : mongo:7
 - Port exposé : 27017 (interne)
 - Rôle : Base de données MongoDB interne à l'application

2.3 Arrêt et maintenance

- Arrêter les conteneurs :
 - `docker compose down`
- Arrêter et supprimer les volumes (réinitialise la BDD) :
 - `docker compose down -v`

3. Justification des choix technologiques

3.1 Langage

- JavaScript
 - Langage orienté Web.
 - Utilisation tant côté client que serveur (via Node.js).
 - Large bibliothèque de modules (via npm).
 - Facilité d'utilisation de code asynchrone (await / async)

3.2 Framework Backend

- Express.js
 - Open Source
 - Rapide à prendre en main mais avec un fonctionnement complet
 - Standard pour API RESTful
 - Simplicité d'utilisation et de mise en place de middlewares

3.3 Framework Frontend

- React
 - Open Source
 - Utilisation de composants réutilisables
 - Simplicité de routage
 - L'un des standards de l'industrie web

3.4 Base de données

- MongoDB
 - Open Source
 - Schémas flexibles et faciles à mettre en place
 - Grosse scalabilité horizontale
 - Facilité et rapidité de mise en place et d'utilisation
 - Utilisation native de Json

3.5 Conteneurisation

- Docker
 - Standard de l'industrie
 - Reproductibilité de l'environnement
 - Isolation des services
- Docker compose
 - Centralise le déploiement de l'application
- Docker file
 - Définit les spécificités de chaque service

3.6 Choix de l'api tierce

- RAWG
 - Base de données énorme (+ 500 000 jeux vidéo)
 - Différents endpoints avec des données complètes
 - Limite d'utilisation largement suffisante pour une utilisation applicative scolaire (20 000 requêtes par mois)

3.7 Authentification

- JWT
 - Sécurisé
 - Simplicité de mise en place
 - Large utilisation dans l'industrie
- Google OAuth
 - Généralité de google (tout le monde a un compte google)
 - Confort utilisateur (évite à l'utilisateur la manipulation de création de compte)
 - Facilité de mise en place du système

4. Diagrammes UML

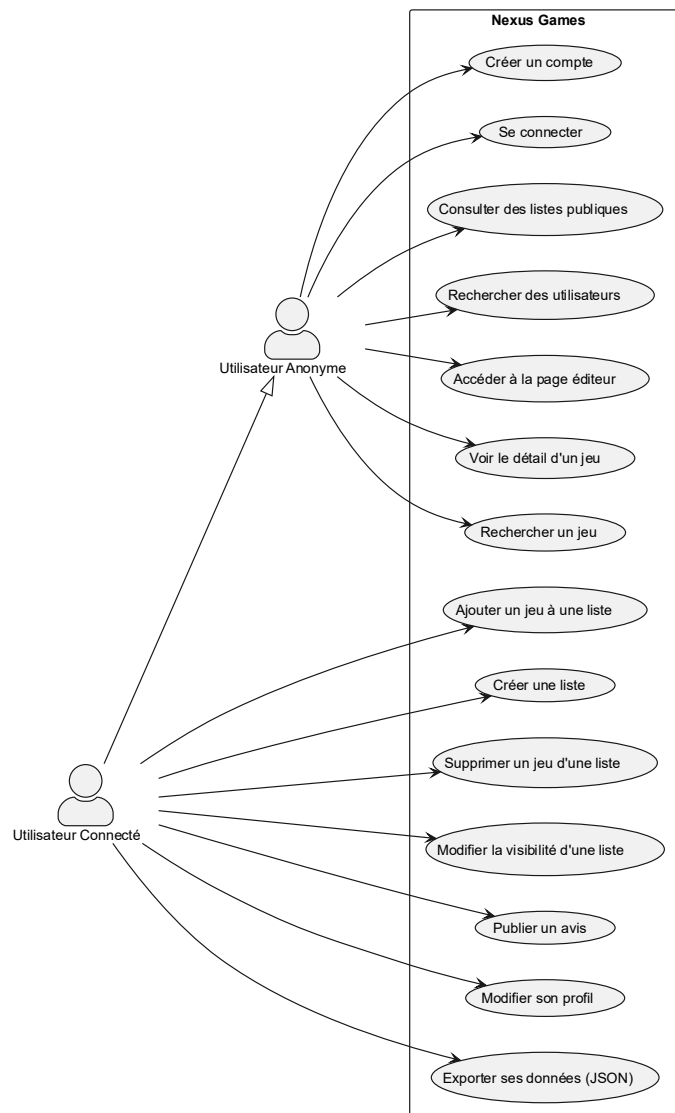
4.1 Diagramme de cas d'utilisation

L'utilisateur anonyme peut :

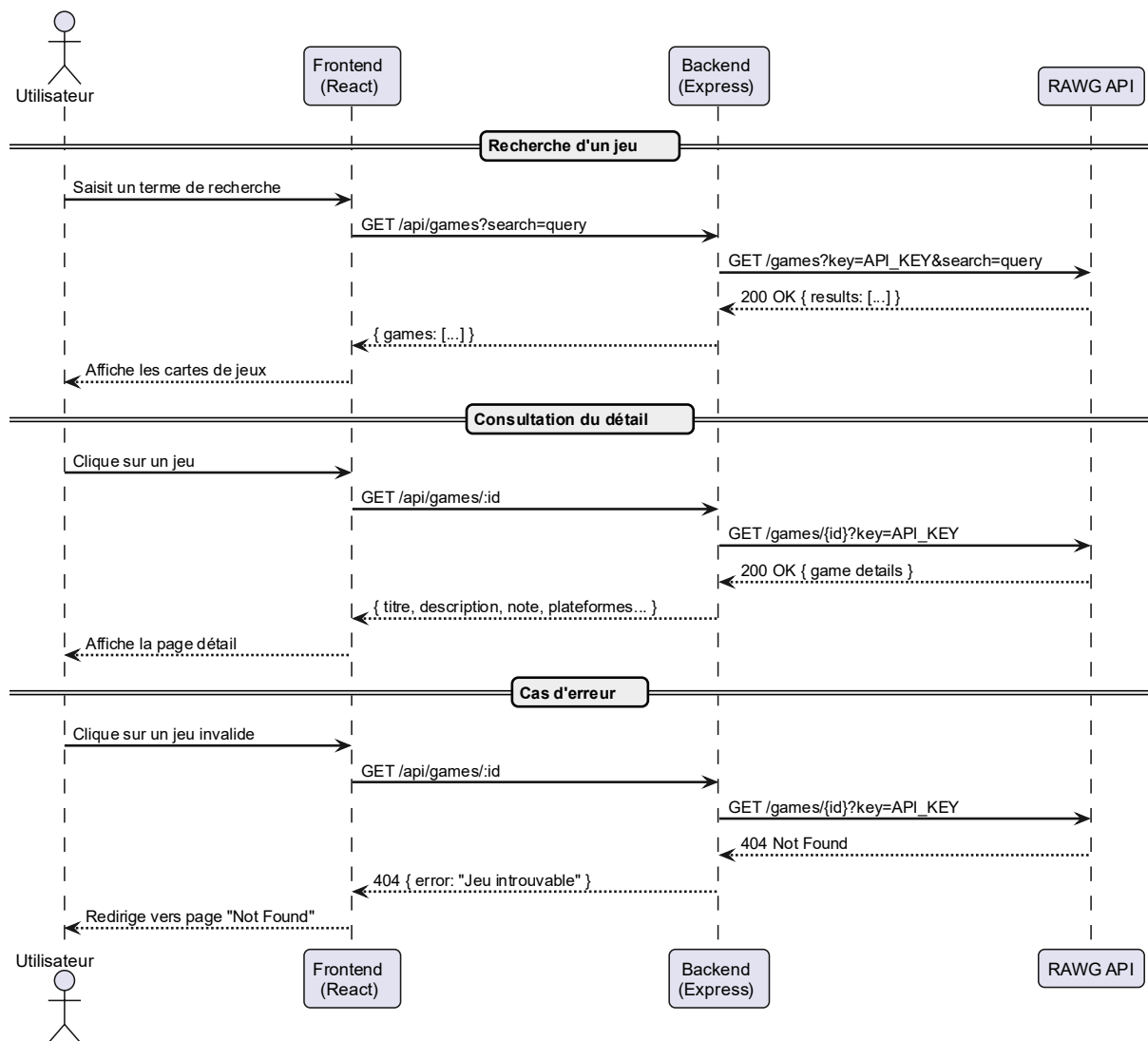
- Créer un compte
- Se Connecter
- Effectuer une recherche
(Jeu / utilisateur / liste)
- Consulter les listes et utilisateurs publics
- Accéder aux détails des jeux et à la page éditeur

L'utilisateur connecté hérite des permissions de l'utilisateur anonyme et peut en plus :

- Créer / Gérer ses listes
- Publier / supprimer ses avis
- Modifier son profil
- Exporter ses données



4.2 Diagramme de séquence — Interaction avec l'API RAWG



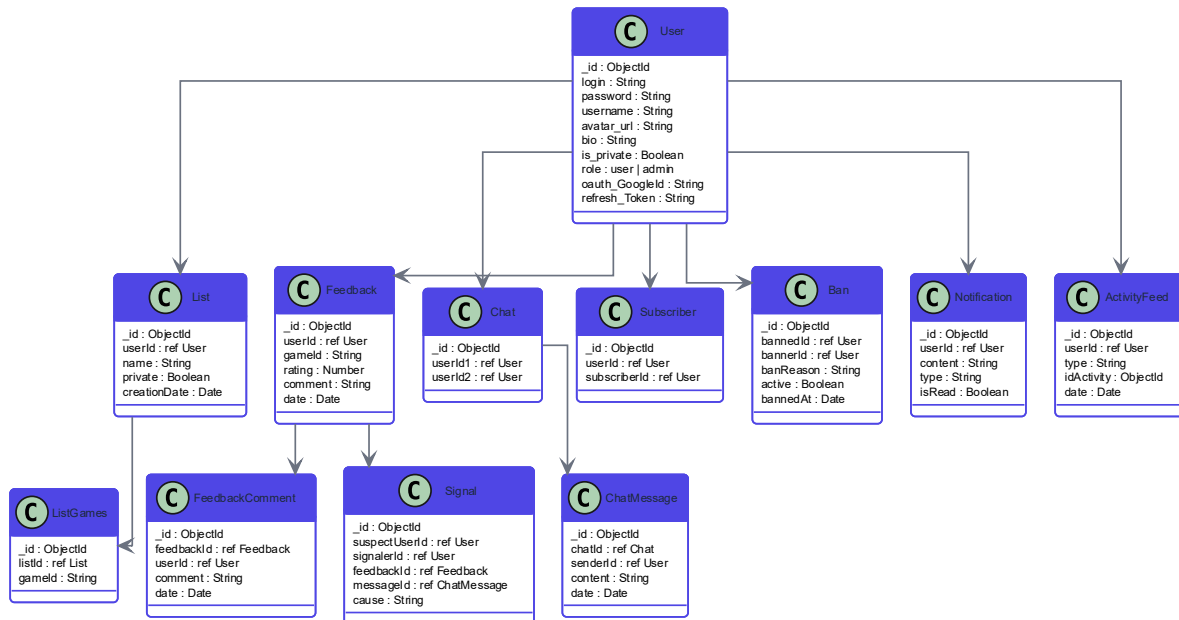
Le frontend ne communique jamais directement avec RAWG.

Toutes les requêtes passent par le backend Express qui agit comme proxy, il enrichit l'appel avec la clé API stockée dans les variables d'environnement, évitant ainsi d'exposer la clé côté client.

Un cache est également mis en place afin d'éviter de répéter les mêmes requêtes en boucle.

5. Schéma de la base de données

La base MongoDB contient 11 collections (11 tables). Les données des jeux ne sont pas stockées localement : seul leur identifiant RAWG (gameId) est stocké, les détails étant récupérés à la volée via l'API.



5.1 Relations entre collections

Collection	Champ(s)	Référence
List	userId	User
ListGames	listId	List
Feedback	userId	User
FeedbackComment	originalFeedbackId	Feedback
FeedbackComment	userId	User
Chat	userId1, userId2	User
ChatMessage	chatId	Chat
ChatMessage	senderId	User
Notification	userId	User
Subscriber	userId, subscriberId	User
ActivityFeed	userId	User
Ban	bannedId, banReason	User
Signal	suspectUserId, signalerId	User
Signal	feedbackId	Feedback

Un utilisateur peut posséder plusieurs listes, publier plusieurs avis, participer à plusieurs conversations et générer plusieurs événements d'activité. Toutes les collections sont donc reliées à User.

5.2 Conception des models

- Table de jonction ListeGames

Les jeux ne sont pas stockés dans la collection List directement. List ne contient que les informations relatives à la liste. Les jeux appartenant à la liste sont stockés dans ListGames, ce qui permet d'interroger ou de supprimer une entrée précise sans réécrire le document parent.

- gameId stocké en tant que string

Dans la base de données les jeux ne sont pas stockés, seul l'identifiant RAWG est persisté dans Feedback et ListGames. Les données du jeu sont récupérées à la volée via l'API au moment de l'affichage.

- Unicité de certains index

Des index de certains models sont uniques dans la base de données :

- Feedback : gameId + userId
 - Un utilisateur ne peut laisser qu'un seul avis par jeu
- ListGames : listId + gameId
 - Un jeu ne peut apparaître qu'une fois dans une liste
- Chat : userId1 + userId2
 - Il ne peut y avoir qu'une seule conversation entre 2 utilisateurs
- Signal : suspectUserId + signalerId
 - Un utilisateur ne peut signaler un certain autre utilisateur qu'une fois
- List : userId + name
 - Un utilisateur ne peut pas avoir 2 listes ayant le même nom

- Signal polyphorme

Un signalement peut viser soit un avis (feedbackId) soit un message (messageId), les deux champs étant optionnels. Cela permet de couvrir plusieurs types de contenus signalables sans multiplier les collections.

- Sparse oauth_GoogleId

Dans le model user, oauth_GoogleId est unique avec Sparse, ainsi les documents sans ce champ ne sont pas inclus dans l'index. Cela évite les conflits d'unicité pour les utilisateurs qui n'ont pas lié de compte Google.